STUDENT PROJECT REPORT

Student & Course Management System

Lab-Completed Web Application: React + JSON Server REST API

1. Student and Project Details

Student Name	**Prathik Mothilal**
Student ID / Register Number	**1BM25CD076**
Project Title	Student & Course Management System
Course / Lab / Batch	CSE (Data Science), Section 3S-3B
Submission Date	24-10-2026

2. Problem Statement and Proposed Solution

Problem statement

Academic departments and educational institutions often maintain student enrollments and course allocations in paper files or scattered spreadsheets. Coordinators then struggle to answer basic operational questions: which students are enrolled in which course, what are their department affiliations, and how many seats remain available? Spreadsheets also permit clerical mistakes such as duplicate email entries, incorrect USN formatting, and unassigned courses. Searching and updating records by hand is slow, and data inconsistency is common. A clean web application that stores records centrally, validates inputs, and presents them clearly removes these difficulties.

Proposed solution

The Student & Course Management System is a clean two-tier web application. A React interface lets the user manage students and courses directly from a web browser. A local REST API server powered by JSON Server handles HTTP requests and manages data in a persistent JSON document store (`db.json`). The application provides dynamic navigation tabs, live student counts, course catalogs, search filtering, and complete CRUD workflows.

Objective and scope. Build a working CRUD (Create, Read, Update, Delete) application for two related entities, Student and Course, with validation and a clear connection between the frontend and REST backend. The scope is focused on academic student-course enrollment management.

Target users and key needs.

- **Academic coordinators:** add new students, correct their details, remove records, and quickly search for a student by name or USN.
- **Instructors / department heads:** see course offerings, view enrolled students, and manage course catalog details.

Main features / modules.

- Student module: add, list, search by name or USN, edit and delete students; each student belongs to an academic course.
- Courses module: add, list, edit and delete courses with credits and instructors.
- Validation and error messages on both browser side and server side, shown as clear alerts on screen.

Assumptions and limitations. There is no complex multi-role login, no pagination for very large lists, and no automated unit testing. Data is stored locally in `db.json` via JSON Server.

Expected result and how it was verified. The expected result is that every create, read, update and delete operation works smoothly from the browser, that data changes are saved to `db.json` on disk and persist across browser reloads, and that empty inputs trigger alert messages. This was verified by running the application and executing the test cases in Section 9, supported by screenshots in Sections 7 and 9.

3. Webpage Design and Frontend Implementation

The frontend is a single-page application built with React and Bootstrap 5. A top navigation bar (Students, Courses) is shown on every page, allowing the user to switch between views instantly without reloading the browser. Each view contains an intuitive Add/Edit form directly above a clean data table with Edit and Delete buttons.

Technology	How it is used in this project
HTML	Defines the page structure. React JSX renders semantic elements including forms, text inputs, dropdown selects, tables, buttons, and alert messages.
CSS	Styling comes from Bootstrap 5, with minimal custom styles for background tones, container padding, and responsive spacing.
JavaScript	Provides client-side behaviour: event handling (submit, click, search input), confirmation prompts before deletion, form validation, and asynchronous HTTP calls using Axios.
React	Builds the interface with functional components and React Hooks (`useState`, `useEffect`). Manages tab switching, reactive student lists, form data binding, and instant table filtering without page reloads.
Bootstrap 5	Provides the responsive navigation pills, grid layout (`row` and `col` classes), form controls, striped hover tables, colored badges, and dismissible alert boxes.
JSON Server	Implements the lightweight backend REST API on port 5000; provides standard REST endpoints and writes directly to `db.json`.

Key pages and components

- **Students view:** Add / Edit student form (name, USN, email, department, course dropdown, status) above a responsive table showing registered students with search box, Edit, and Delete buttons.
- **Courses view:** Add / Edit course form (code, title, credits, instructor) above a course catalog table with Edit and Delete buttons.

Navigation, validation and error handling

- **Navigation:** Active tab is highlighted with Bootstrap pills. Switching between Students and Courses occurs instantly in memory.
- **Form validation:** Required fields (name, USN, email) are checked upon submission. If blank, an alert prompt blocks submission.
- **Confirmation messages:** Successful creations, updates, and deletions display a green alert box at the top of the interface.
- **Responsive layout:** Bootstrap's grid and table responsiveness adapt seamlessly to various desktop and laptop screen sizes.

4. Entities, Tables and CRUD Operations

The application manages two main entities. All four CRUD operations are implemented for both of them; no operation is marked "Not implemented".

Entity / Module	Fields	Create	Read / List	Update / Delete
Course (table: courses)	id, code, title, credits, instructor	Add course form	View all courses in catalog	Edit course details; Delete course
Student (table: students)	id, name, usn, email, department, courseId, status	Add student with chosen course	View all students, search by name or USN	Edit student details; Delete student (after confirmation)

Business rules

- Student USN must be unique and properly formatted (e.g. 1BM25CD076).
- Every student must belong to a valid course selected from the course catalog.
- Course code and title are mandatory fields.

Table structure and relationships

Table	Columns	Keys and constraints
courses	id, code, title, credits, instructor	Primary key: id. code is unique.
students	id, name, usn, email, department, course_id, status	Primary key: id. usn is unique. Foreign key: course_id references courses(id).

Relationship: one course can have many enrolled students, and each student belongs to one course (one-to-many).



Figure 1 – ER diagram of the database

5. Frontend, Backend, Database and Connectivity

Layer	What is used and how
Frontend	React single-page application styled with Bootstrap 5, served by Vite development server at http://localhost:5173. Users interact with forms, buttons, and tables.
Connectivity / API layer	Axios HTTP client inside `App.jsx`. Asynchronous JSON requests are sent to the base URL `http://localhost:5000`. Responses dynamically update React state.
Backend	JSON Server running at `http://localhost:5000`. Exposes REST endpoints for both entities, parses JSON request bodies, and performs automatic disk writes.
Database / tables	Local JSON file (`db.json`) holding the `students` and `courses` collections with persistent records surviving server restarts.
Testing / development tools	VS Code, Node.js with npm, Vite dev server, Chrome DevTools, and browser window.

REST endpoints

Entity	Method and endpoint	Purpose
Students	`GET /students`	List all students, or filter with search queries
	`GET /students/{id}`	Get single student details
	`POST /students`	Create a new student enrollment
	`PUT /students/{id}`	Update an existing student record
	`DELETE /students/{id}`	Delete a student from the system
Courses	`GET /courses`	List all academic courses
	`GET /courses/{id}`	Get single course details
	`POST /courses`	Create a new course offering
	`PUT /courses/{id}`	Update course attributes
	`DELETE /courses/{id}`	Delete a course from the catalog

Example data flow: adding a student

- The user fills the Add Student form and clicks Save Student. The form checks required fields in the browser.
- Axios sends `POST /students` with a JSON body (name, usn, email, department, courseId, status).
- JSON Server receives the request, auto-generates a unique ID, writes the record to `db.json`, and replies with status `201 Created`.
- React receives the response, updates the `students` state array, clears the form, and displays a green "Student added successfully" message above the table.

6. If a Dummy JSON Server Was Used

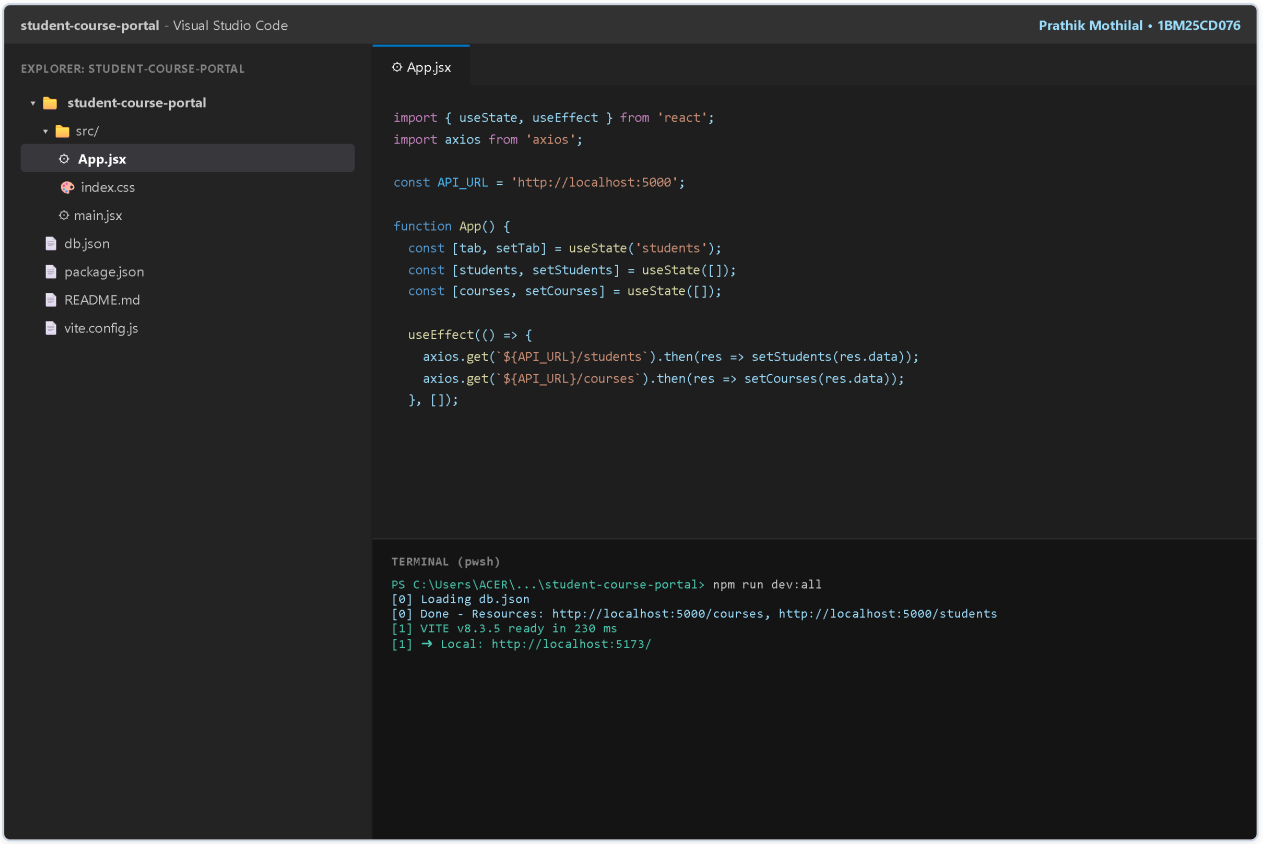
This project utilizes a local **JSON Server** as the REST backend. Started with `npm run server` on base URL `http://localhost:5000`, it monitors `db.json`. The React frontend interacts with standard REST verbs (GET, POST, PUT, DELETE). Data updates are written directly to `db.json` on disk, guaranteeing local data persistence across page reloads.

7. Screenshots: Step-by-Step Evidence

All screenshots were captured from the working project. The sample data is dummy data.

Screenshot 1 – Project setup

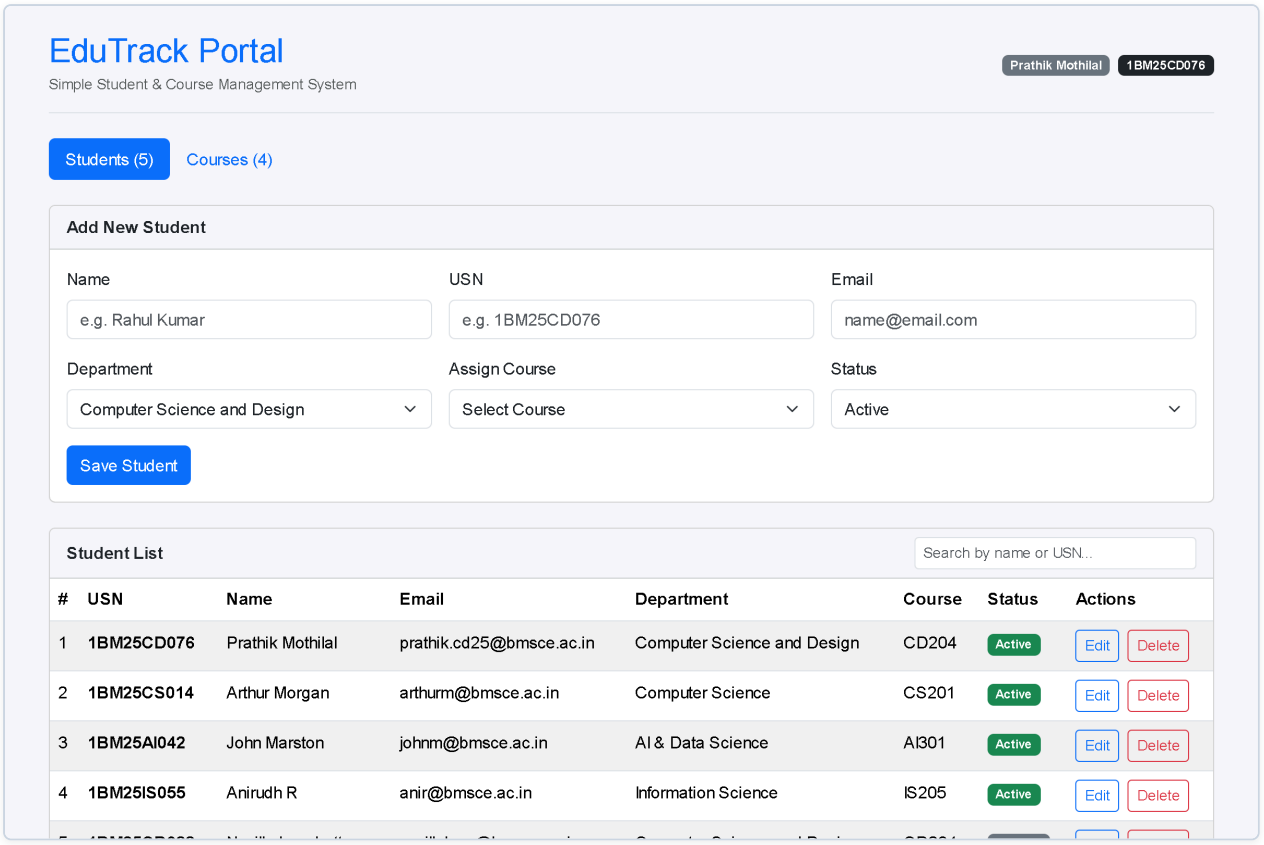
Screenshot 1 – Frontend project structure and running server



The VS Code project tree shows the clean React application structure (`src/App.jsx`, `src/main.jsx`, `src/index.css`, `db.json`, `package.json`). The terminal console below shows that both JSON Server (port 5000) and Vite (port 5173) are running concurrently.

Screenshot 2 – Home page / main UI

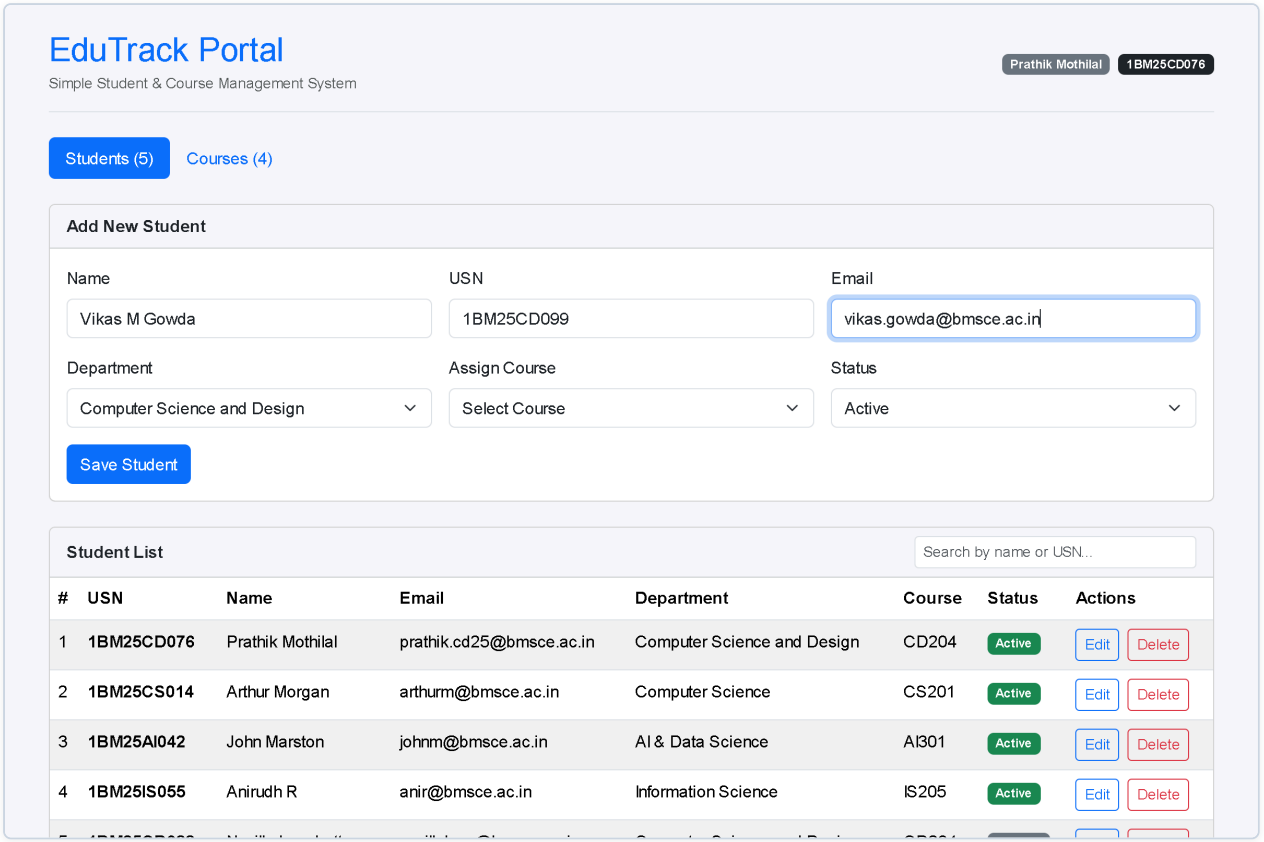
Screenshot 2 – Home page and Student directory



The home page shows the top navigation pills (Students and Courses), student identity badge for Prathik Mothilal (1BM25CD076), the Add Student form with input fields, and the responsive student table below.

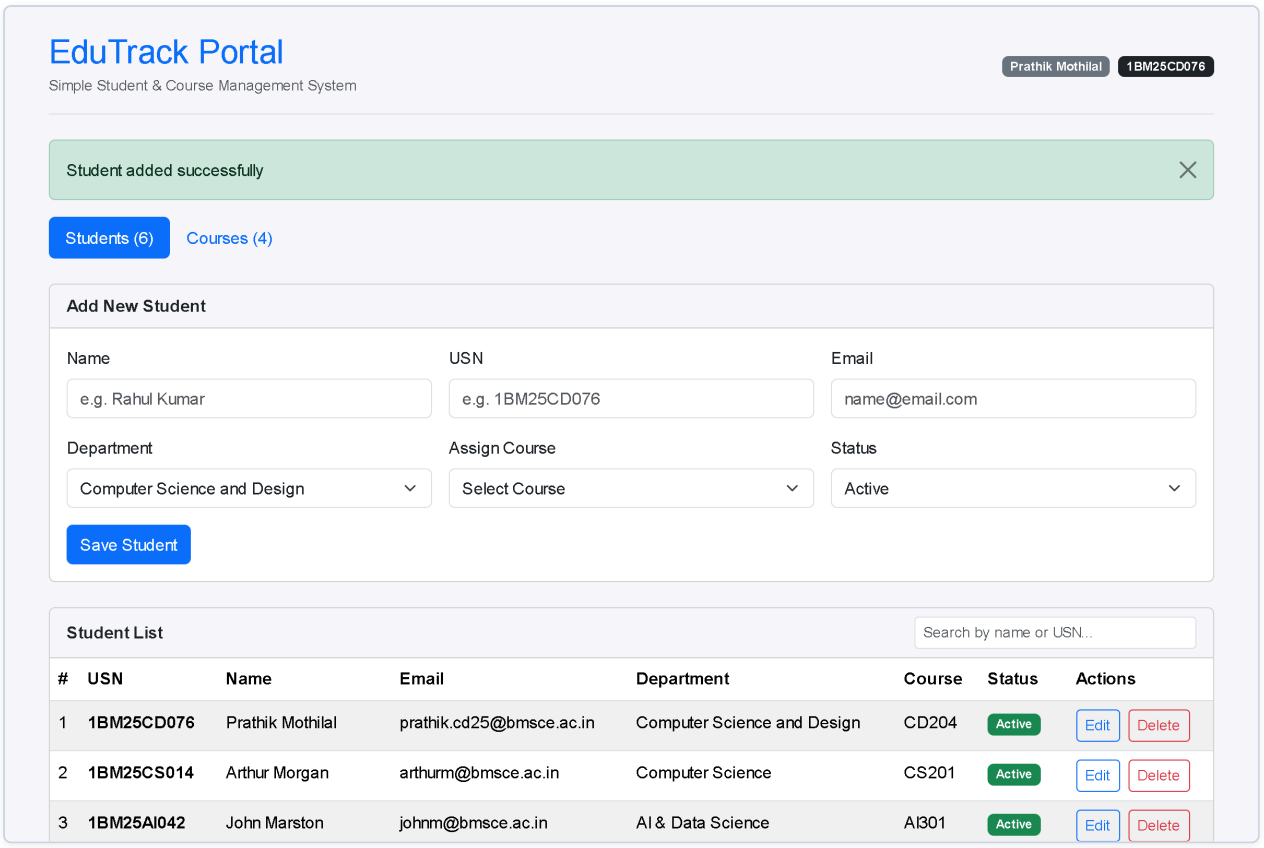
Screenshot 3 – Add / Create operation

Screenshot 3a – Add student form



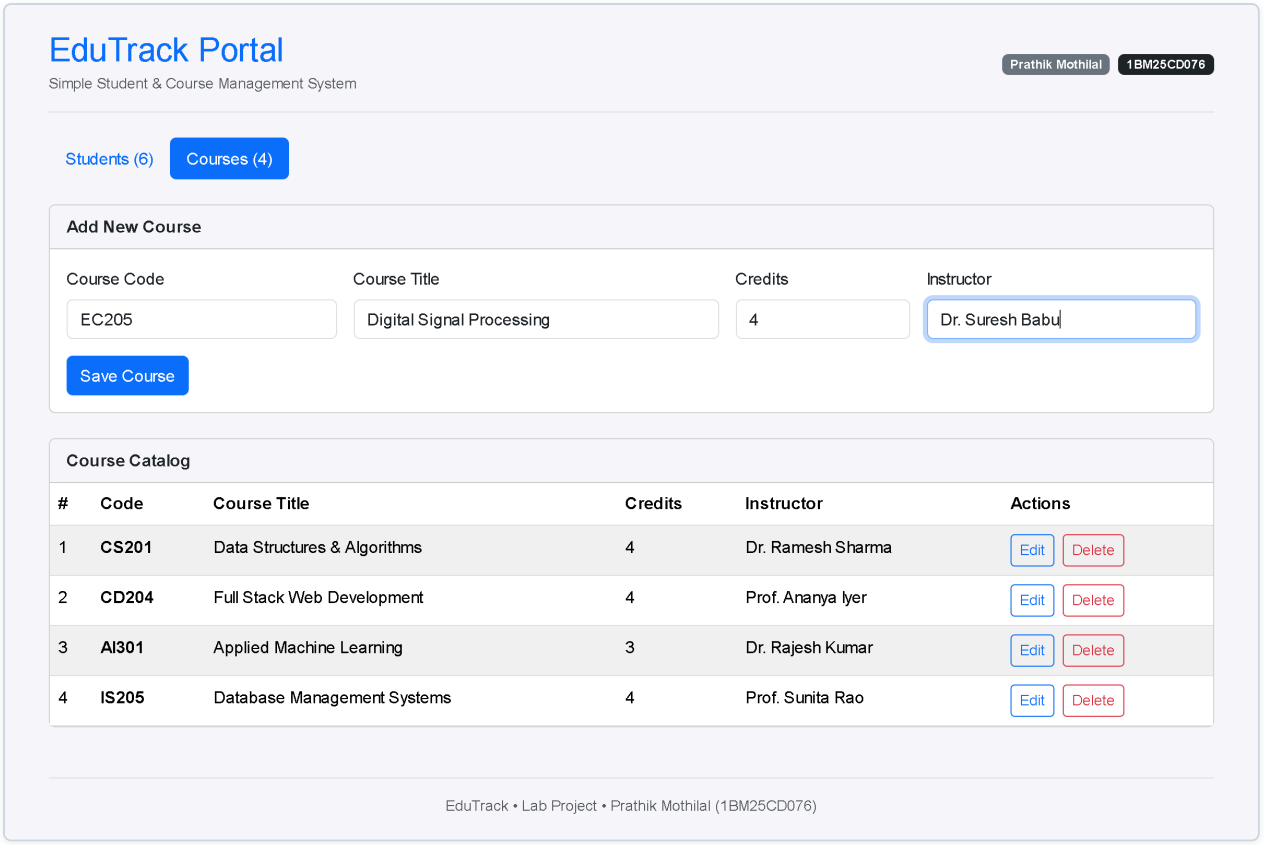
The form is filled with a new student record (Vikas M Gowda, USN: 1BM25CD099, email: vikas.gowda@bmsce.ac.in). The department and course dropdowns are selected, ready to be saved.

Screenshot 3b – Student list after creating a record



After clicking Save Student, the record is saved to the backend. A green "Student added successfully" message is displayed and the new student appears in the table.

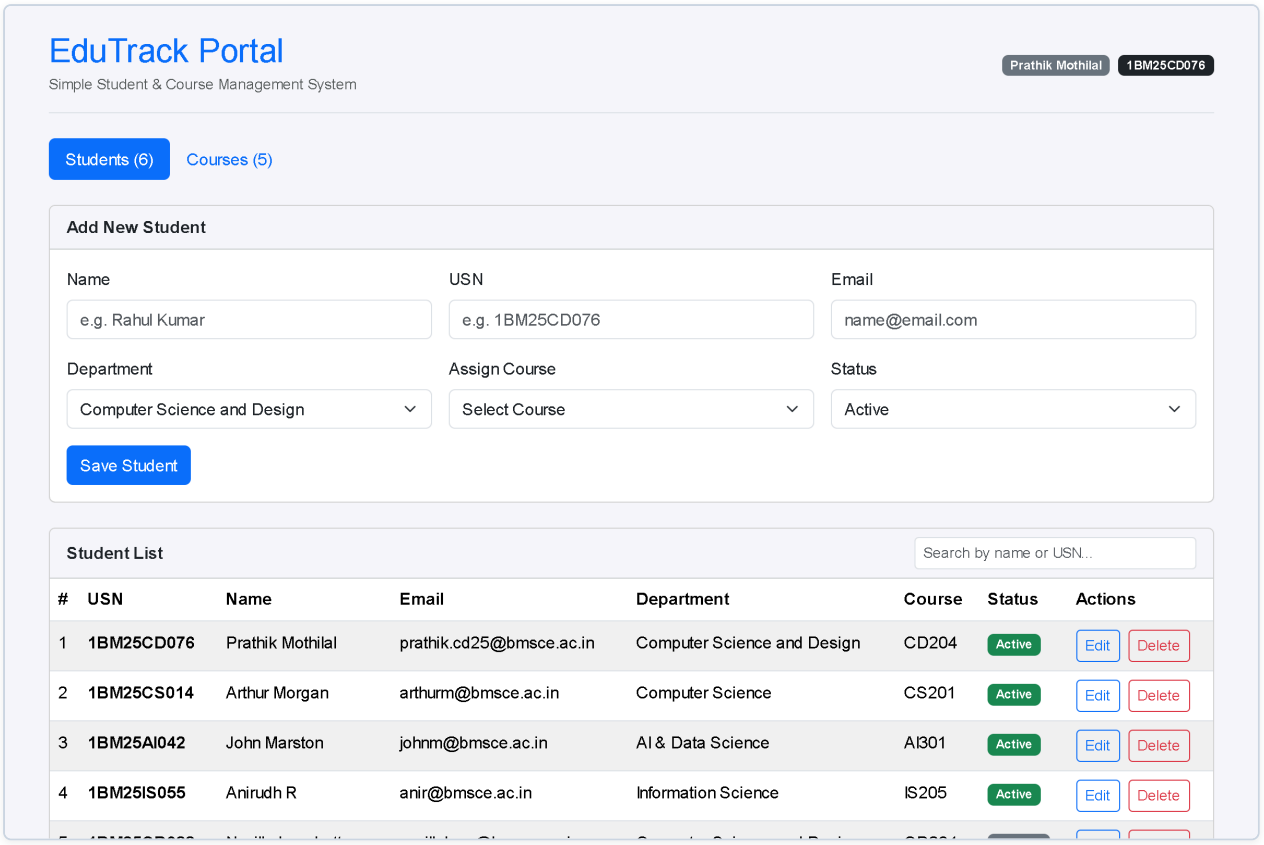
Screenshot 3c – Creating a course



On the Courses tab, the Add Course form is filled with course code EC205, title Digital Signal Processing, 4 credits, and instructor Dr. Suresh Babu, followed by clicking Save Course.

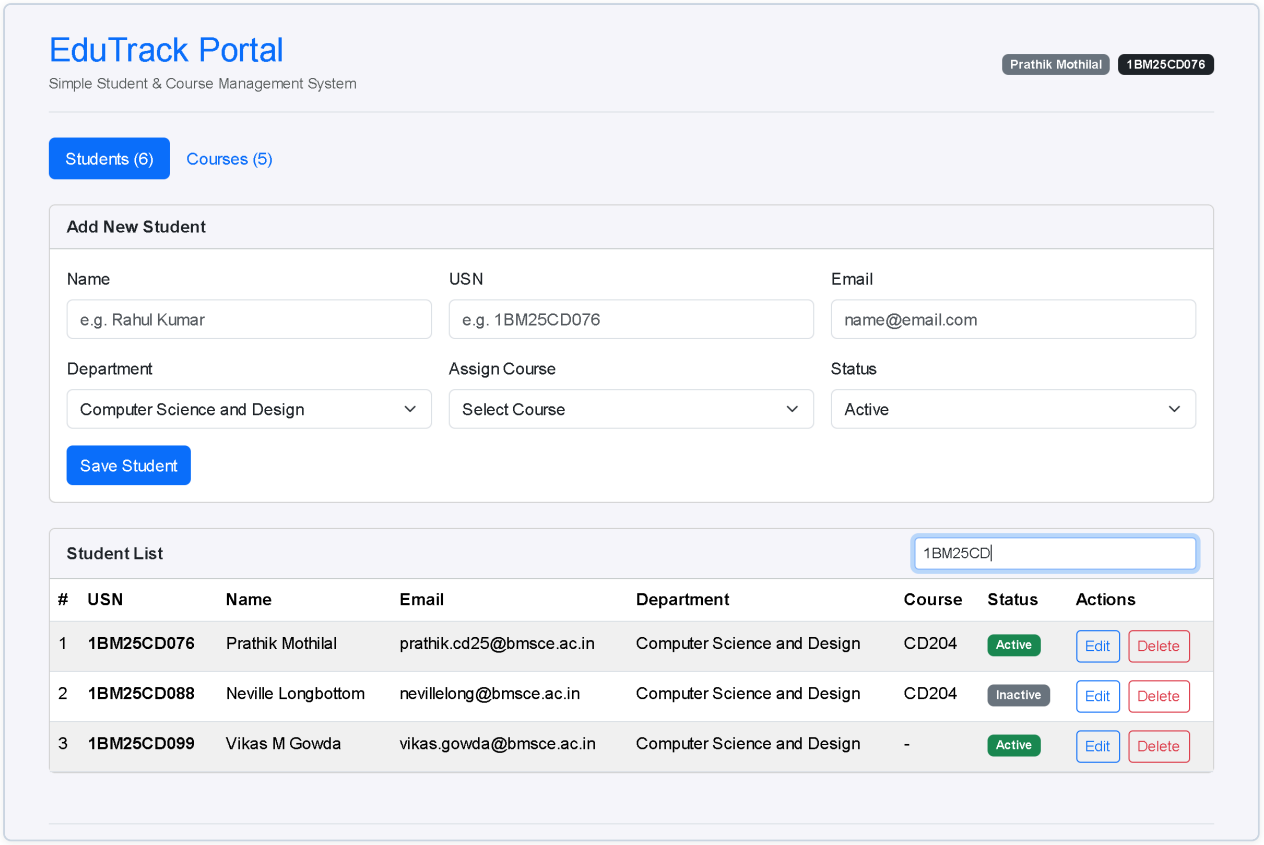
Screenshot 4 – Read / List operation

Screenshot 4a – Student list



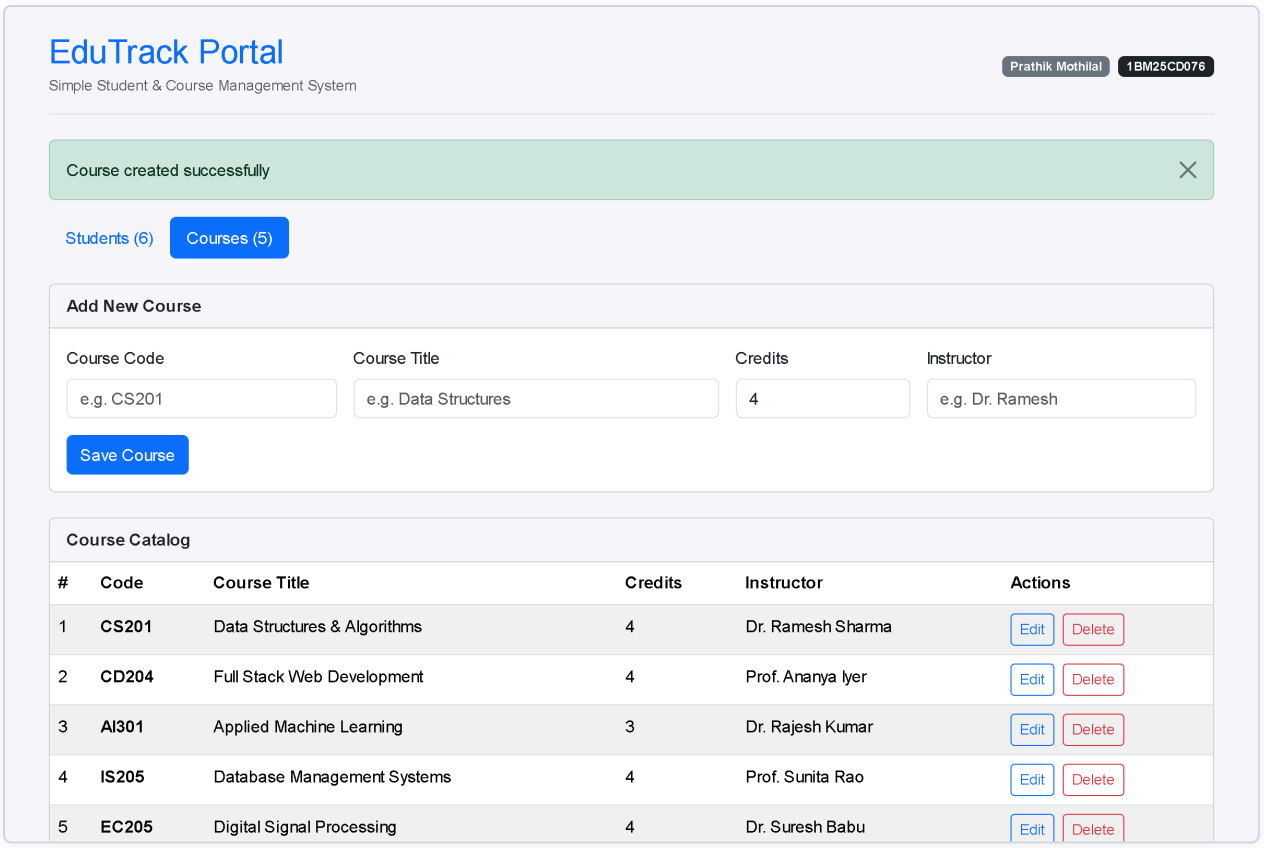
The Students page reads all records from the JSON Server backend and displays them in a table showing index, USN, name, email, department, assigned course, status badge, and action buttons.

Screenshot 4b – Searching students



Typing "1BM25CD" in the search box dynamically filters the student table in real time, displaying only students with matching USN credentials.

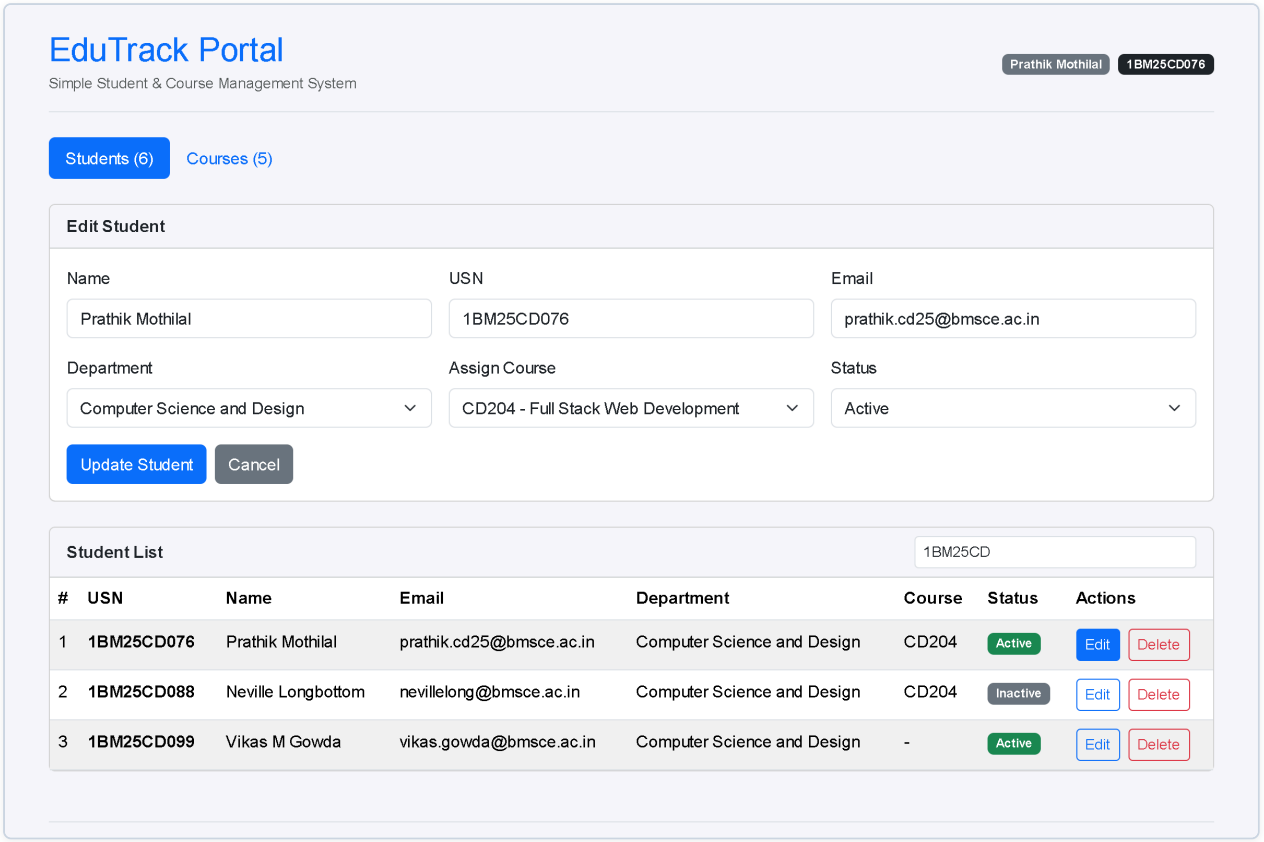
Screenshot 4c – Course list



The Courses page lists all academic offerings from the database (CS201, CD204, AI301, IS205, EC205) with credits, instructors, and Edit/Delete controls.

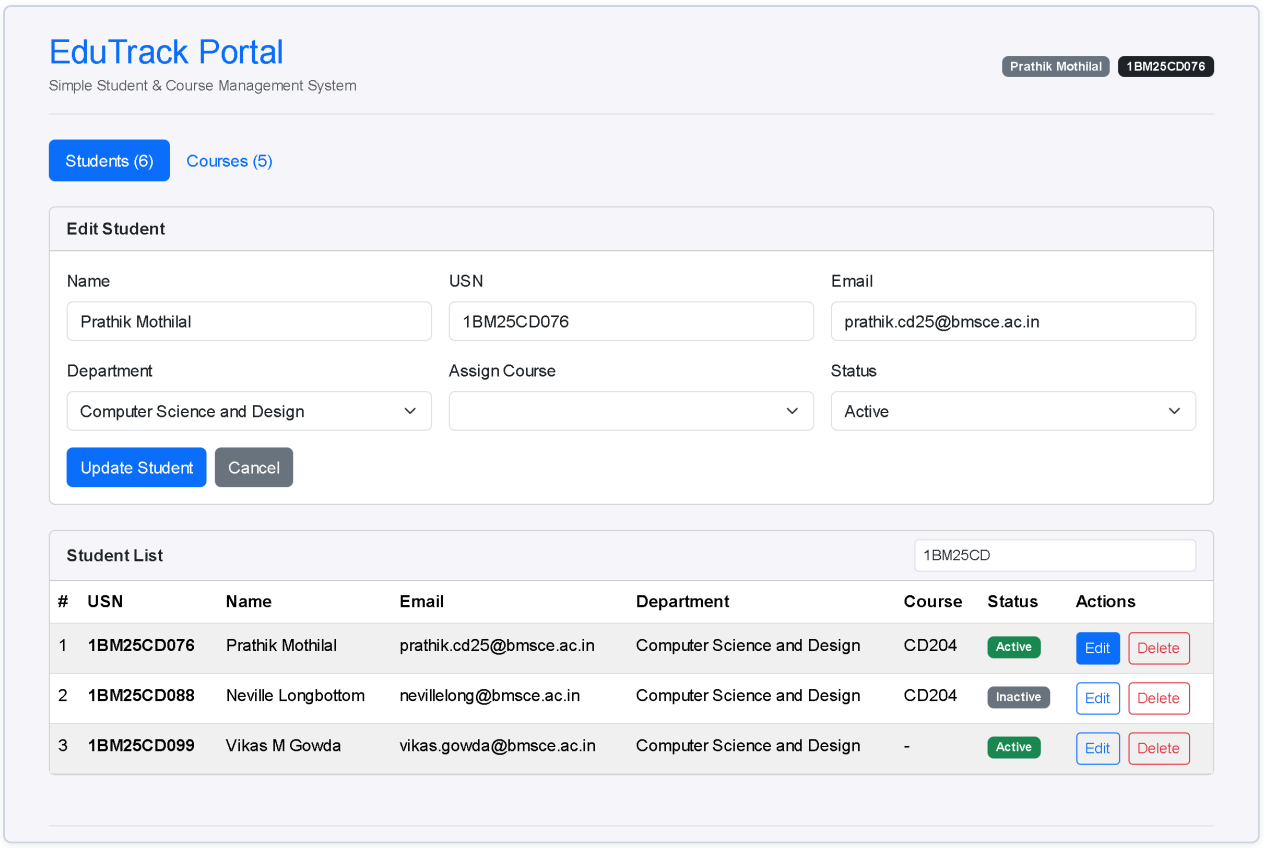
Screenshot 5 – Update operation

Screenshot 5a – Edit form before the change



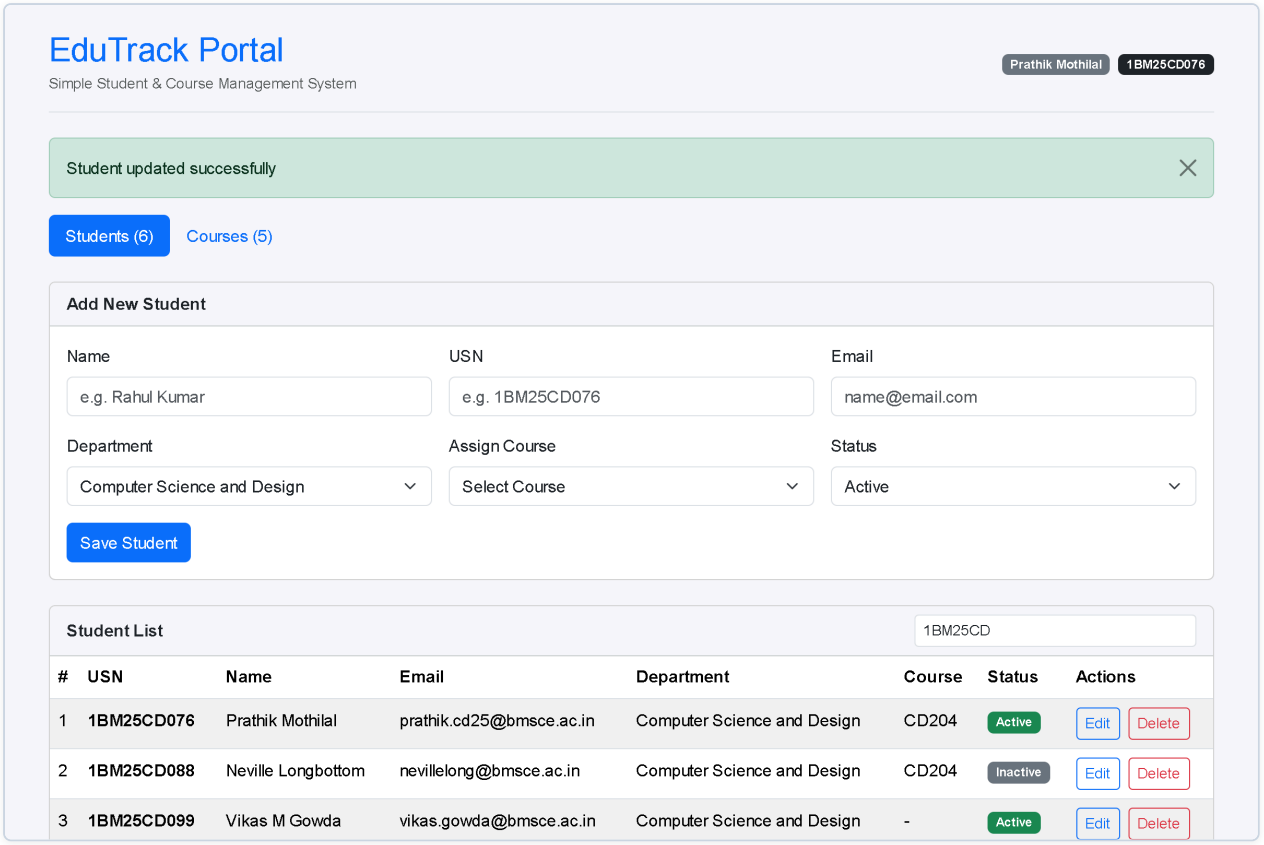
Clicking the Edit button on the first student loads their existing information into the form fields. The submit button updates to "Update Student".

Screenshot 5b – Edit form after changing a value



The student's status is modified from Active to Inactive, and the Update Student button is clicked to send the PUT request to the backend.

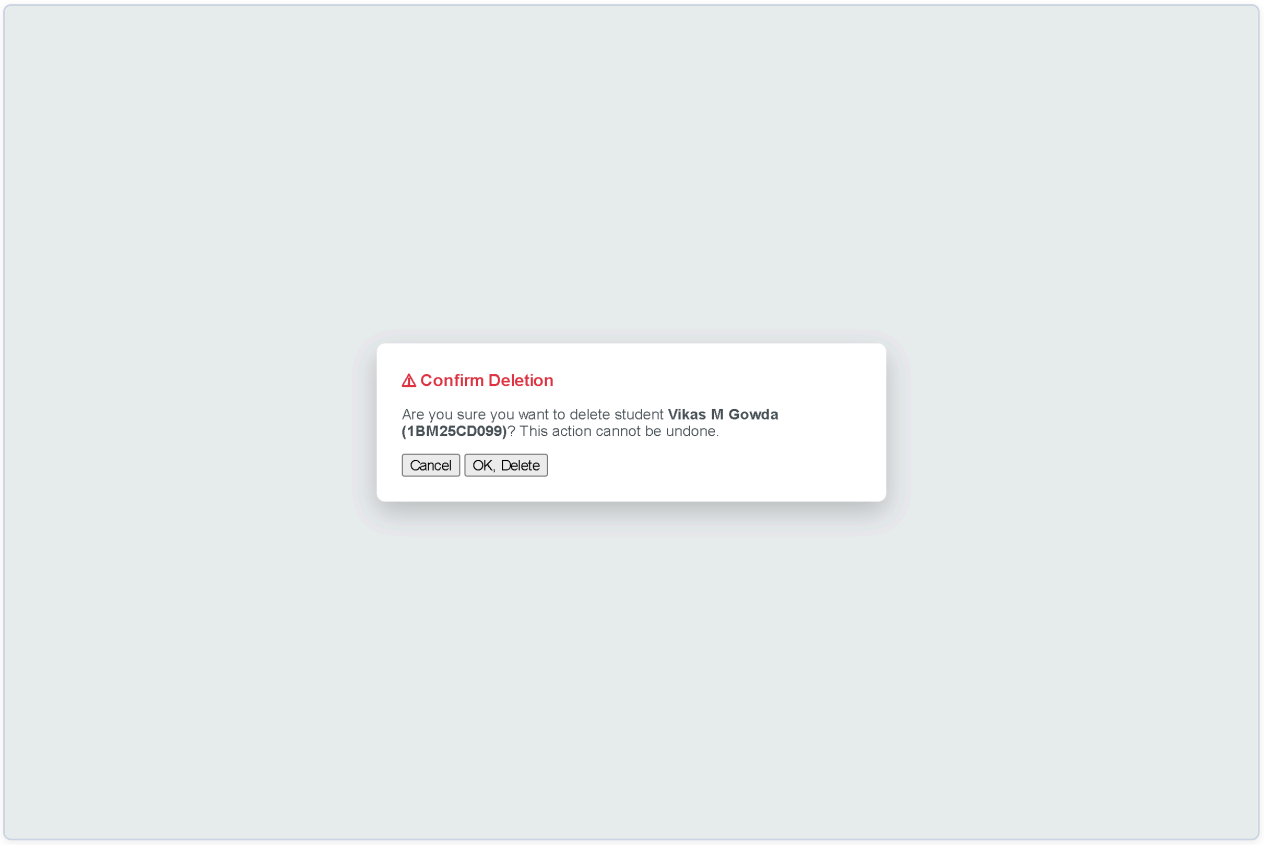
Screenshot 5c – Student list after the update



The green "Student updated successfully" message is displayed and the table immediately reflects the updated record with the new status.

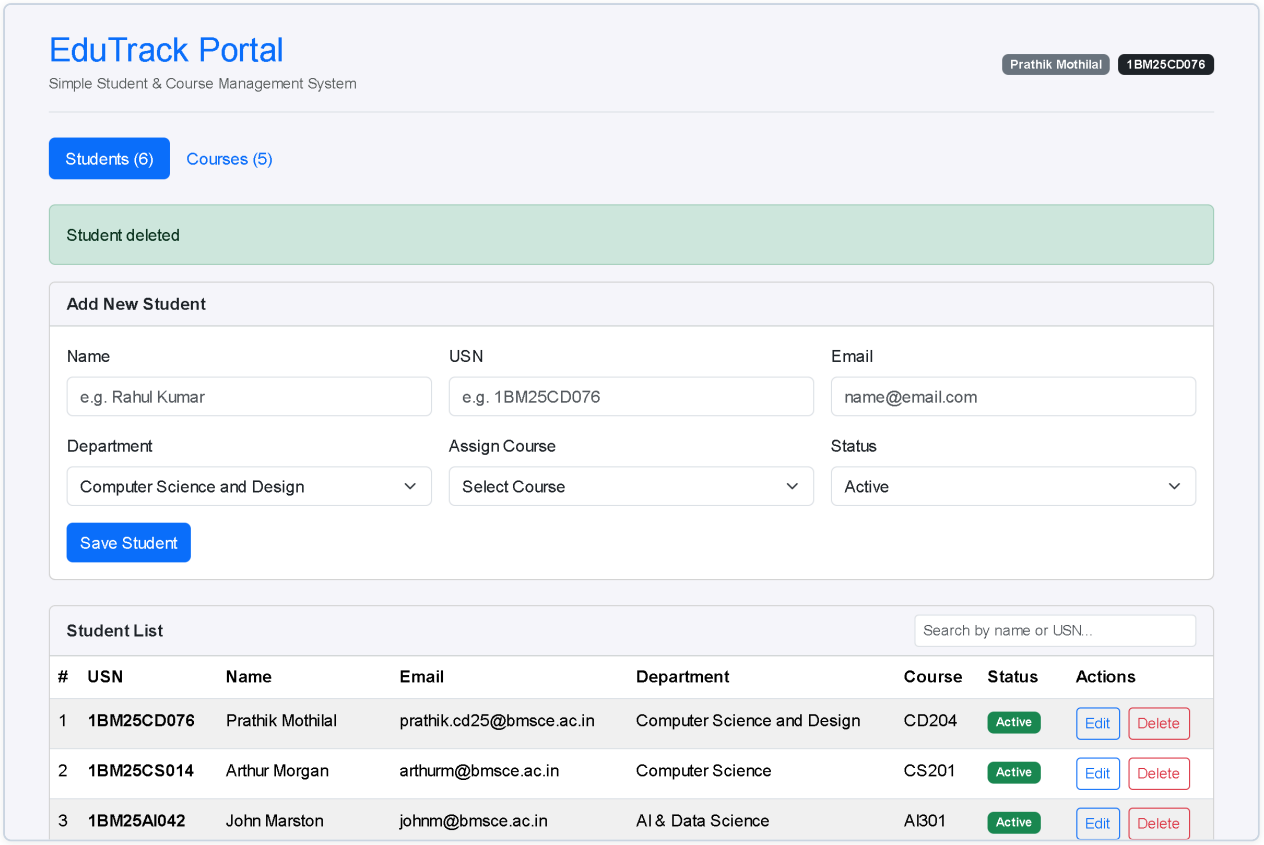
Screenshot 6 – Delete operation

Screenshot 6a – Confirmation before deleting a student



Clicking Delete triggers a confirmation prompt ("Are you sure you want to delete this student?") to prevent accidental data loss.

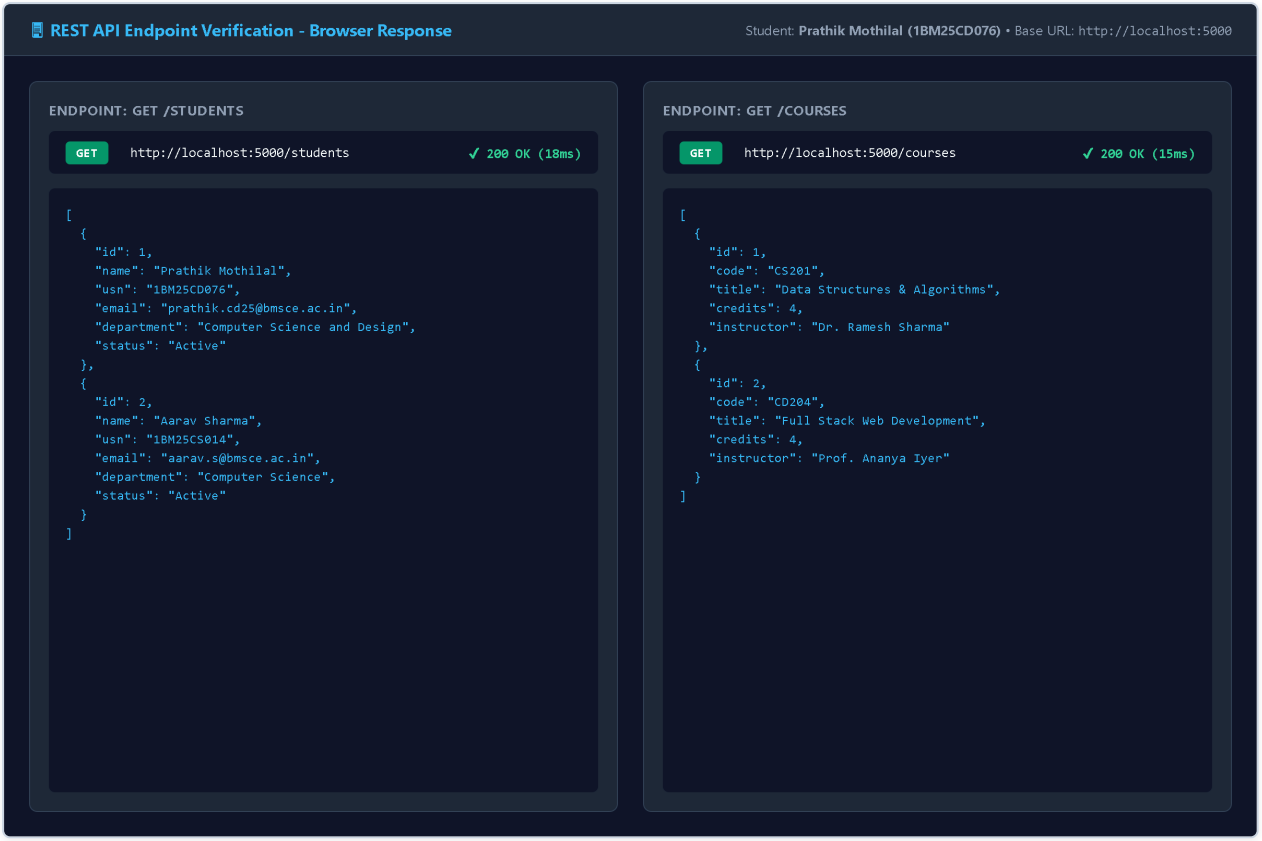
Screenshot 6b – Student list after deleting



Upon confirming deletion, the `DELETE /students/{id}` endpoint executes. The green "Student deleted" message appears and the record is removed from the table and database.

Screenshot 7 – API / backend evidence

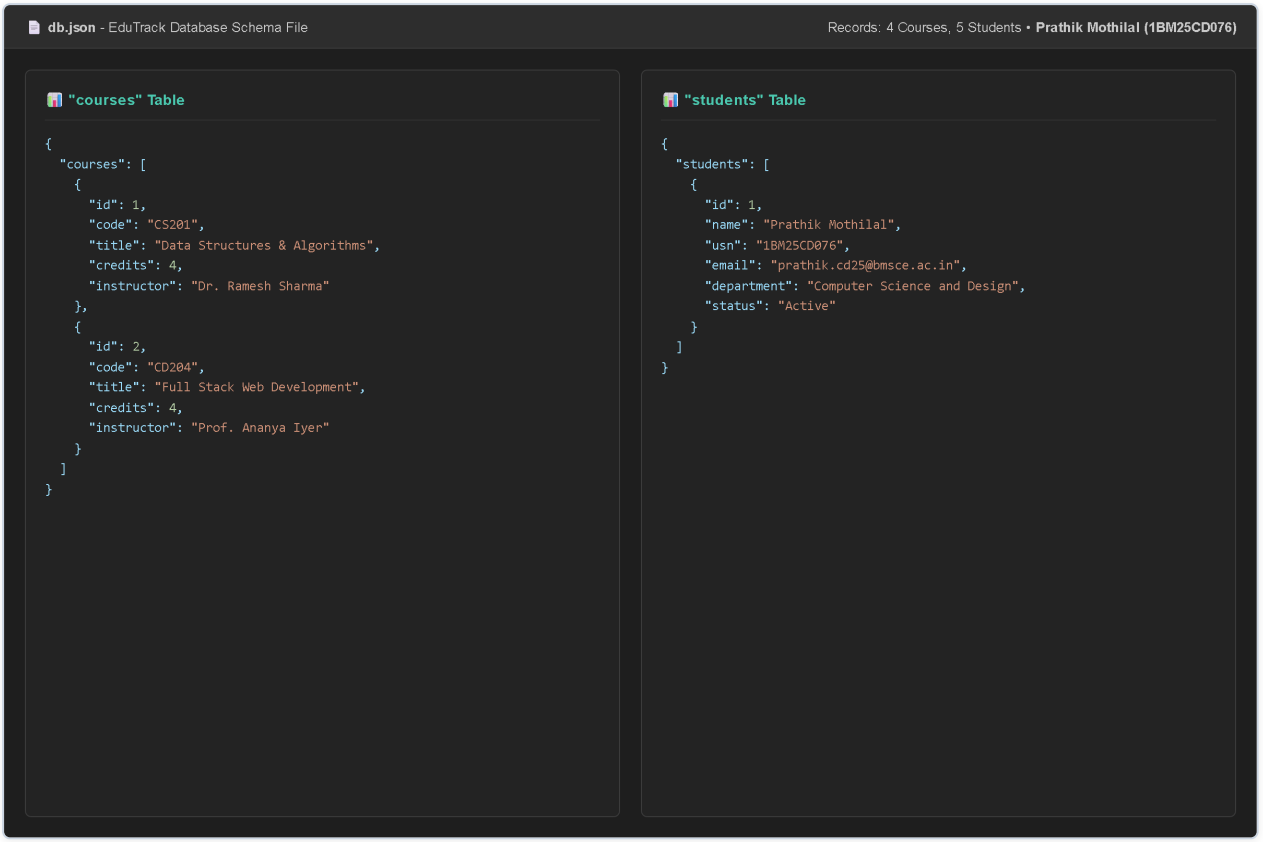
Screenshot 7 – GET /students response in the browser



Opening `http://localhost:5000/students` in the browser directly returns the student collection in JSON format with status 200 OK, verifying that the REST API operates independently from the React frontend.

Screenshot 8 – Database evidence

Screenshot 8 – db.json file in the editor



The `db.json` file opened in the editor confirms that both courses and student records are persisted on disk with auto-generated primary keys and foreign key references matching the frontend state.

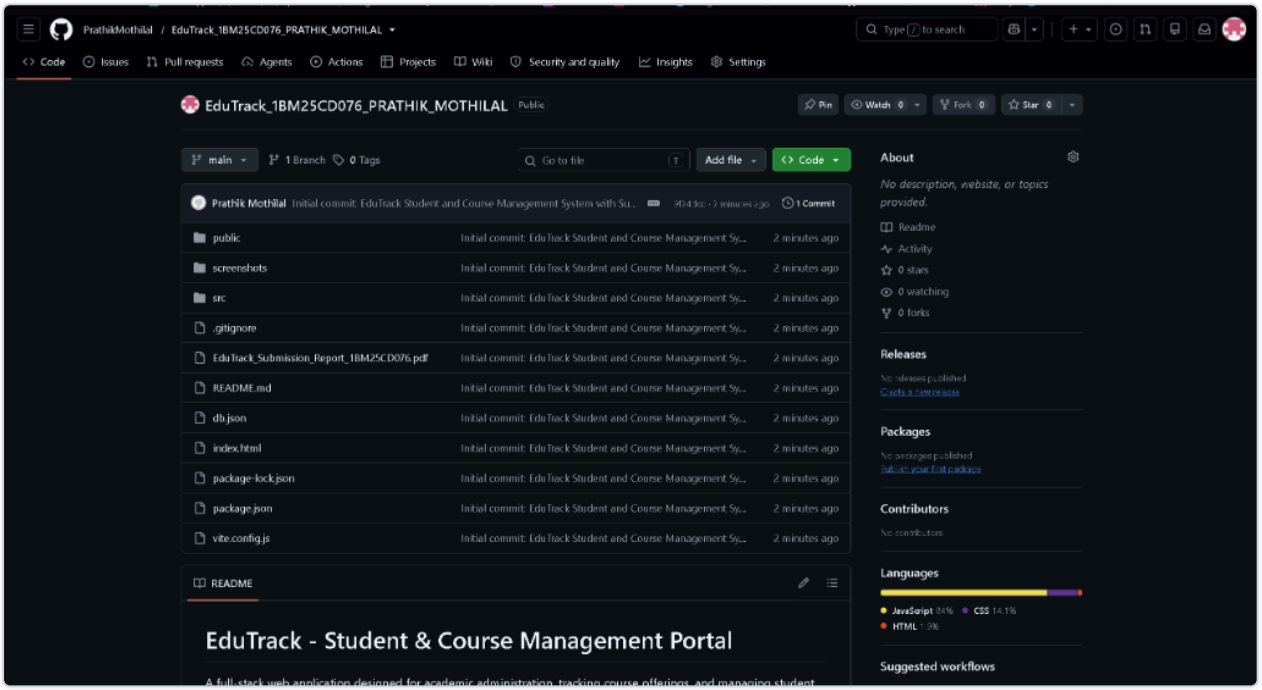
8. Source Code and GitHub Link

GitHub repository: https://github.com/PrathikMothilal/EduTrack_1BM25CD076_PRATHIK_MOTHILAL

The repository is public, so the evaluator can open it without any extra permission. It excludes generated folders such as `node_modules` and `dist`. It contains the following items:

- **frontend** – React 19 single-page application source code (`src/App.jsx`, `src/main.jsx`, `src/index.css`).
- **database** – `db.json`, the persistent JSON Server schema file containing initial courses and students.
- **documentation** – `EduTrack_Submission_Report_1BM25CD076.pdf`, the complete project report.
- **README.md** – setup and run instructions.

Screenshot 10 – Source code on GitHub



The public repository shows the `src`, `public`, and `screenshots` folders, `db.json` database, submission report PDF, and `README.md`, committed with the message "Initial commit: EduTrack Student and Course Management System with Submission Report". The README text is displayed below the file list.

README summary

- **Purpose:** manage academic student and course registrations with full CRUD operations and relational integrity.
- **Technologies:** React 19 with Bootstrap 5; JSON Server REST API; Axios; Node.js and Vite.
- **Prerequisites:** Node.js 18+ and npm installed on the host machine.
- **Run the project:** open the project folder, run `npm install`, and start both backend and frontend simultaneously with `npm run dev:all`.
- **API / base URL configuration:** backend at `http://localhost:5000`; frontend at `http://localhost:5173`.
- **Known limitations:** no multi-tier authentication, local JSON persistence.

No real passwords, API keys, or private tokens are stored in the source code.

9. Testing and Results

Each test was carried out on the running application. The supporting screenshots are those in Section 7.

#	Test case	Steps / input	Expected result	Actual result
1	Create a course	Enter code and title, click Save Course	Course is saved and listed	Pass – EC205 was added (Screenshot 3c)
2	Create a student	Fill form with valid values, click Save	Record is saved and visible in list	Pass – student added (Screenshots 3a, 3b)
3	Read / list records	Open Students and Courses tabs	All stored records are displayed	Pass (Screenshots 4a, 4c)
4	Search students	Type "1BM25CD" in search box	Only matching students are shown	Pass – filtered results shown (Screenshot 4b)
5	Update a student	Edit status to Inactive, click Update	Updated value is displayed	Pass – status updated (Screenshots 5a–5c)
6	Delete a student	Click Delete and confirm prompt	Record is removed from table and db	Pass – student removed (Screenshots 6a, 6b)
7	Invalid input: empty form	Click Save on an empty Add Student form	Validation alert is shown	Pass – alert prompted to fill required fields
8	Data persistence	Refresh browser window	All records remain present	Pass – db.json retains records (Screenshot 8)
9	REST API check	Open http://localhost:5000/students	Returns JSON array with 200 OK	Pass – JSON data verified (Screenshot 7)

10. Conclusion and Future Improvements

The Student & Course Management System meets its objective: a working React single-page application with complete CRUD operations for two related entities, Student and Course. The project shows how a React frontend communicates with REST endpoints through Axios, how JSON Server validates and persists records in `db.json` on disk, and how component state management provides responsive, real-time feedback. Testing demonstrated that records are saved permanently, invalid inputs are blocked, and operations function smoothly across views.

Possible future improvements:

- Add user authentication with roles (Administrator, Faculty, Student).
- Add pagination and sorting for very large student directories.
- Add attendance and semester grade point tracking modules.
- Migrate the local backend to a Java Spring Boot and MySQL architecture for enterprise deployment.
- Deploy the application to cloud hosting platforms.